

Operating System & Computer Organisation

Answers to 38 Questions with Diagrams

Prepared from the uploaded course material: Chapters 1-6, Unit 2, Unit 3 (Fundamentals of OS) and Practice Exercise 2. Figures marked (*Source: ...*) are taken from those PDFs; the remaining diagrams (user/kernel mode, Resource Allocation Graph, safe/unsafe states) were drawn for this document.

Q1. Differentiate between User mode and Kernel mode.

The processor can execute in two modes of operation. **User mode** is the less-privileged mode in which application programs normally run. **Kernel mode** (also called system mode or control mode) is the more-privileged mode in which the kernel of the operating system runs. The mode is recorded in the Program Status Word (PSW) as a supervisor/user bit.

Basis	User mode	Kernel mode
Privilege level	Less privileged, restricted	More privileged, unrestricted
Who runs here	Application programs, editors, compilers, browsers	Operating-system kernel, interrupt handlers, device drivers
Instructions	Privileged (machine-level) instructions are not allowed	All instructions, including privileged ones, may be executed
Memory access	Only its own address space; protected areas cannot be touched	Can access protected memory areas, including the OS and other processes
Hardware access	Indirect, only through system calls	Direct access to I/O devices and hardware
Effect of a crash	Only that program is affected	May crash the whole system
Switch to the other mode	System call, interrupt or trap moves the CPU into kernel mode	Return from the kernel (after servicing) moves the CPU back to user mode

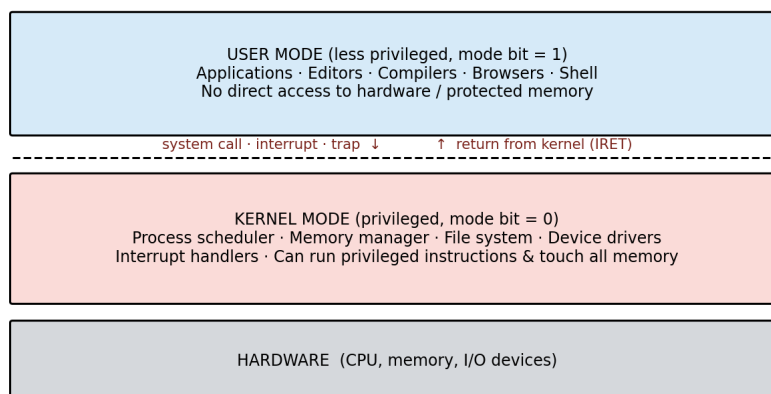


Fig. 1 – User mode and kernel mode (drawn for this document)

Q2. Explain the concept of interrupts and discuss why interrupts are required in a computer system.

An **interrupt** is a mechanism by which other modules (I/O, memory, timer) may interrupt the normal sequencing of the processor. When an interrupt occurs, the processor suspends the program it is executing, runs a program called the **interrupt handler** (generally part of the OS) to service the device, and then resumes the interrupted program.

Why interrupts are required

- **To improve processor utilisation.** Most I/O devices are much slower than the processor. Without interrupts the processor has to pause and wait for the device (for example a printer).
- Without interrupts, after issuing an I/O command the program must either wait or repeatedly *poll* the device status. Polling wastes processor time (busy waiting).
- With interrupts the processor issues the I/O command, continues executing other instructions and is interrupted when the device is ready. This is especially valuable when the I/O wait is long.
- Interrupts are the basis of multiprogramming: after the handler completes, control may be given to a different program.
- They let the OS handle clock (time-slice) events, I/O completion, memory faults, errors (traps) and supervisor calls (see Q27).

Interrupt cycle

After executing each instruction the processor checks for interrupts. If none is pending, it fetches the next instruction of the current program. If an interrupt is pending, it suspends the current program and executes the interrupt-handler routine.

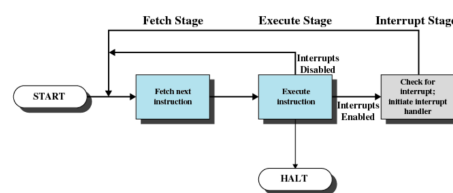


Figure 1.7 Instruction Cycle with Interrupts

Fig. 2 – Instruction cycle with interrupts (Source: Chapter 1, Fig. 1.7)

Simple interrupt processing

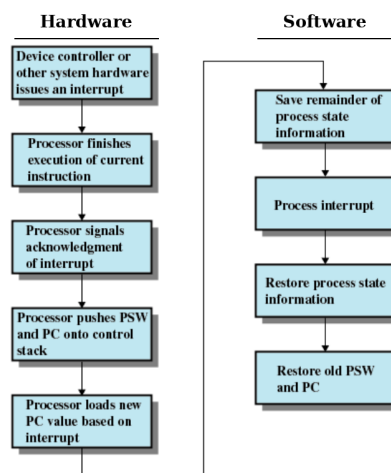


Fig. 3 – Simple interrupt processing: hardware actions (left) and software actions (right) (Source: Chapter 1, Fig. 1.10)

Hardware: the device signals an interrupt; the processor finishes the current instruction; acknowledges the interrupt; pushes the PSW and PC onto the control stack; loads the new PC from the interrupt vector. **Software:** the handler saves the rest of the process state, services the interrupt, then restores the process state and the old PSW and PC.

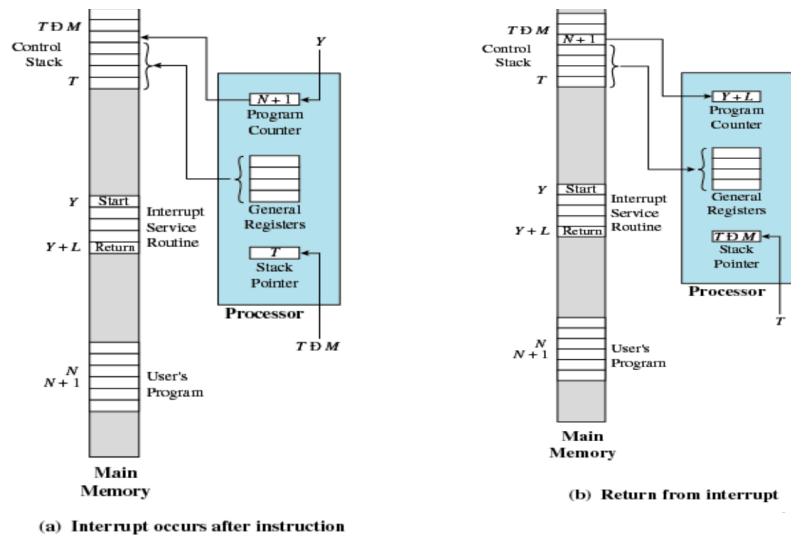


Fig. 4 – Changes in memory and registers for an interrupt (Source: Chapter 1, Fig. 1.11)

Multiple interrupts

- **Disable interrupts** while one is being processed – interrupts are handled strictly in sequence.
- **Define priorities** – a higher-priority interrupt can interrupt a lower-priority handler (nested handling).

Q3. Describe the various services provided by the operating system. Also, define the goals of operating systems.

An operating system is a program that controls the execution of application programs and acts as an interface between applications and hardware. It also acts as a resource manager for the processor, memory and I/O devices.

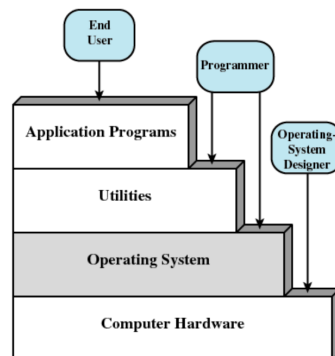


Fig. 5 – Layers and views of a computer system (Source: Chapter 2, Fig. 2.1)

Services provided by the OS

Service	Description
Program development	Editors, debuggers and other utilities that help programmers create programs
Program execution	Loads the program into memory, prepares I/O and files, and runs it
Access to I/O devices	Hides the details of each device behind a uniform interface
Controlled access to files	Manages file structure, naming, storage and protection
System access	Controls access to the system and its resources (especially in shared systems)
Error detection and response	Handles hardware errors (memory error, device failure) and software errors (arithmetic overflow, access to forbidden memory); the OS may end the program, retry or report the error
Accounting	Collects usage statistics, monitors performance, helps plan enhancements and is used for billing

Goals of an operating system

Goal	Meaning
Convenience	Makes the computer more convenient to use
Efficiency	Allows system resources (CPU, memory, I/O) to be used in an efficient manner
Ability to evolve	Permits effective development, testing and introduction of new system functions, hardware upgrades and fixes without interfering with service

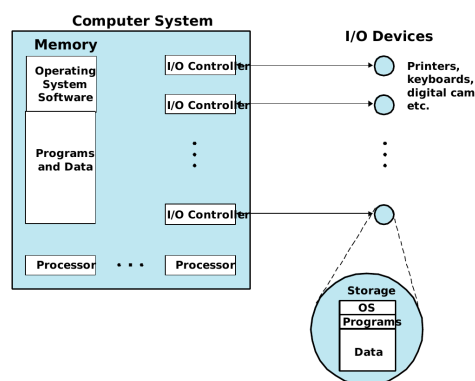


Figure 2.2 The Operating System as Resource Mana

Fig. 6 - The operating system as resource manager (Source: Chapter 2, Fig. 2.2)

Q4. What are the main components of a computer system? Explain their functions.

At the top level a computer consists of four structural components: the **processor (CPU)**, **main memory**, **I/O modules** and the **system bus** that interconnects them.

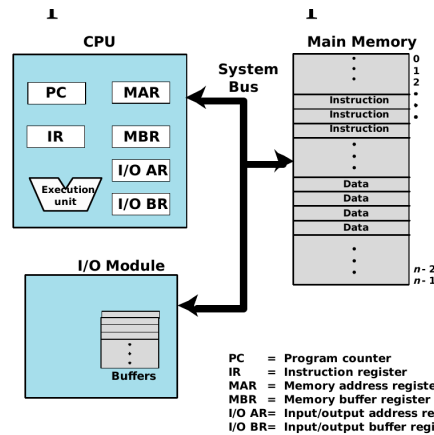


Figure 1.1 Computer Components: Top-Level View

8

Fig. 7 - Computer components: top-level view (Source: Chapter 1, Fig. 1.1)

Component	Function
Processor (CPU)	Controls the operation of the computer and performs data-processing functions. It fetches instructions from memory, decodes and executes them. Contains the ALU/execution unit, control unit and registers (PC, IR, MAR, MBR, I/O AR, I/O BR, general registers).
Main memory	Stores data and instructions; it is volatile and is organised as a sequence of numbered locations (words). Addressed by the processor through the MAR/MBR.
I/O modules	Move data between the computer and external devices (disk, keyboard, printer, network). Each has internal buffers and address/buffer registers.
System bus	Provides communication among the processor, main memory and I/O modules (see Q10).

Q5. Define process states. Design a 5-state process model using a proper state transition diagram.

A **process** is a program in execution. During its life it is not in the same condition forever: it changes **state** according to events in the system. A **process state** describes what the process is doing at a given instant (for example executing, waiting for I/O or waiting for the CPU). The OS tracks the state in the process control block.

Five-state model

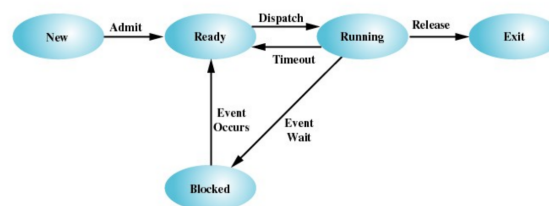


Figure 3.6 Five-State Process Model

Fig. 8 - Five-state process model (Source: Chapter 3, Fig. 3.6)

Transition	Event / meaning
New → Ready (Admit)	OS admits the process to the pool of executable processes (loaded in memory, placed in the ready queue)
Ready → Running (Dispatch)	Scheduler selects the process and gives it the CPU
Running → Ready (Timeout / Interrupt)	Time slice expires or a higher-priority process arrives
Running → Blocked (Event wait)	Process requests I/O or waits for an event or resource

Transition	Event / meaning
Blocked → Ready (Event occurs)	The awaited I/O or event completes
Running → Exit (Release)	Process finishes or is aborted

The same model appears in Unit 2 with the names *Admit*, *Dispatch*, *Interrupt*, *I/O or event wait*, *I/O or event completion* and *Terminated* (equivalent to Exit). A detailed description of each state is given in Q12.

Q6. What is a Process Control Block (PCB)? Draw the diagram of PCB and explain each block of PCB in detail.

The **PCB** is a data structure created and managed by the operating system for every process. It is created when the process is created and deleted when it terminates. It contains all the information needed to identify and control the process, and it allows the OS to support multiple processes by saving and restoring their state.

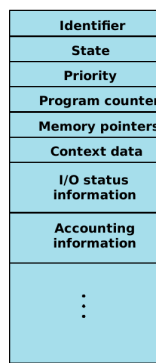


Figure 3.1 Simplified Process Con

Fig. 9 – Simplified process control block (Source: Chapter 3, Fig. 3.1)

Field	Explanation
Identifier (PID)	Unique number identifying the process; may also hold parent PID and user ID
State	Current state: new, ready, running, blocked/waiting, exit (needed for scheduling)
Priority	Scheduling preference over other processes; some systems keep default, current and highest-allowable values
Program counter	Address of the next instruction to be executed; saved at a state change so execution can resume
Memory pointers	Pointers to the program code and data, plus shared memory blocks; also segment/page tables for virtual memory
Context data	Contents of the processor registers (data, address, control/status registers, stack pointers) saved when the process is not running
I/O status information	Outstanding I/O requests, devices assigned to the process, list of open files
Accounting information	CPU time and clock time used, time limits, account numbers

Grouping used in the text book

- **Process identification** – process ID, parent process ID, user ID.
- **Processor state information** – user-visible registers, control and status registers (PC, condition codes, status/mode flags) and stack pointers.
- **Process control information** – scheduling and state information, data structuring (queue and parent-child links), interprocess communication flags, process privileges, memory management, resource ownership and utilisation.

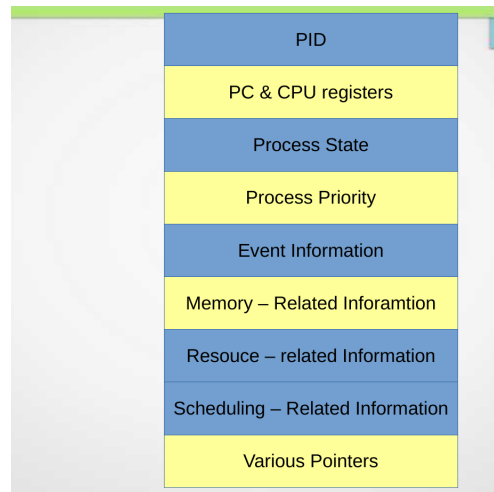


Fig. 10 – PCB as shown in Unit 2: PID, PC & CPU registers, process state, priority, event information, memory-, resource- and scheduling-related information, various pointers (Source: Unit 2, p. 41)

Unit 2 view of the fields: *Event information* – the event a blocked process is waiting for; *Resource-related information* – resources allocated, e.g. open files; *Scheduling-related information* – waiting time, time executed last time; *Various pointers* – pointers to the parent process, child processes and the address space of the process.

Q7. What is a thread? Explain the need for threads in an operating system.

A **thread** is a flow of execution through the process code, with its own program counter, a set of registers holding its current working variables, and a stack containing its execution history. It is also called a **lightweight process** and is the unit of dispatching. Every thread belongs to exactly one process, and all threads of a process share that process's address space, open files and resources. Resource ownership belongs to the *process*; scheduling/execution belongs to the *thread*.

Each thread has: an execution state (running, ready, etc.), a saved thread context when not running, its own execution stack, some per-thread static storage for local variables and access to the memory and resources of its process. Its information is kept in a **Thread Control Block (TCB)**, which is smaller than a PCB (TID, PC, registers, state, priority, event information, scheduling information, pointer to owner thread, TCB pointer).

Need for threads

- **Faster creation** – a thread takes much less time to create than a process.
- **Faster termination** – less time to terminate a thread.
- **Faster switching** – context switching between two threads of the same process is cheaper (less state to save).
- **Easy communication** – threads share memory and files, so they can exchange data without invoking the kernel (no IPC system calls).
- **Better use of multiprocessors** – different threads of one process can run on different processors at the same time.
- **Application structure** – foreground/background work, asynchronous processing, speed of execution and modular program design. Example: a word processor uses one thread for formatting text and another for processing input; browser tabs can be separate threads.
- **Better responsiveness** – a blocked thread need not stop the rest of the application.

Q8. Differentiate between Multiprogramming and Multiprocessing.

Basis	Multiprogramming	Multiprocessing
Meaning	Several programs are kept in memory and the single processor switches among them	The system has two or more processors that execute processes at the same time
Number of CPUs	One	Two or more
Execution	Interleaved: only one process executes at any instant	Overlapped: truly parallel execution of several processes
Main aim	Keep the CPU busy; when a job waits for I/O the CPU switches to another job (better utilisation)	Higher throughput/performance, availability and incremental growth
Hardware	Ordinary uniprocessor	Multiprocessor / multicore (e.g. SMP)
Failure	CPU failure stops the system	Failure of one processor does not halt the machine; it continues at reduced performance
Complexity	Needs scheduling, memory protection and I/O interrupts	Also needs synchronisation, cache coherence and multiprocessor scheduling

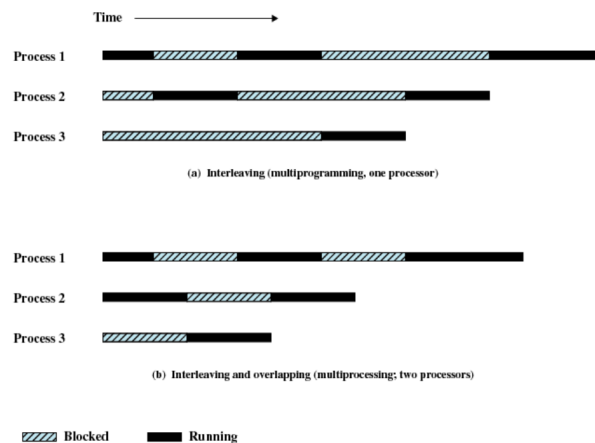


Figure 2.12 Multiprogramming and Multiprocessing

Fig. 11 – (a) multiprogramming: interleaving on one processor; (b) multiprocessing: interleaving and overlapping on two processors (Source: Chapter 2, Fig. 2.12)

Q9. Explain the role of Main Memory and I/O Modules in a computer system.

Main memory

- Holds the **programs and data** that the processor is currently using; the processor can fetch instructions and operands only from here (or from cache/registers).
- Made of semiconductor technology and is **volatile**; organised as numbered locations (0 ... n-1). Each location holds an instruction or data.
- Accessed through the **MAR** (gives the address of the next read/write) and the **MBR** (holds the data being written or the data just read).
- Sits between fast registers/cache and slow secondary storage in the memory hierarchy, giving a balance between cost, capacity and speed.
- The OS divides it among processes (process isolation, protection and virtual memory).

I/O modules

- Provide the interface between the processor/memory and **external devices** (keyboard, printer, disk, network).

- Have internal **buffers**, plus an **I/O address register (I/O AR)** and **I/O buffer register (I/O BR)**, so data can be exchanged at the device's speed.
- Hide device-specific details from the processor and convert between device and system signal formats.

I/O technique	How it works
Programmed I/O	I/O module performs the action; the processor repeatedly checks the status bits until the operation completes (no interrupts)
Interrupt-driven I/O	Processor is interrupted when the module is ready; it saves context and runs the handler. No needless waiting, but every word still passes through the processor
Direct Memory Access (DMA)	Processor grants the I/O module authority to read/write memory directly; a whole block is transferred and an interrupt is sent at the end, so the processor is free meanwhile

Q10. What is a System Bus? Explain its purpose and the types of information transferred through it.

The **system bus** is the communication pathway (a set of shared wires/lines) that connects the processor, main memory and I/O modules (Fig. 7). It lets the components exchange addresses, data and control signals.

Purpose

- Provides a shared path so that any component can communicate with any other.
- Allows the processor to read instructions/data from memory and write results back.
- Carries data between I/O modules and the processor or memory.
- Carries signals that coordinate who may use the bus and what operation is to be performed.

Bus lines	Information carried
Address bus	Address of the memory location or I/O port to be accessed (from MAR / I/O AR). Its width decides the maximum directly addressable memory, e.g. 24 address bits → 16 MB (as in Practice Exercise 1).
Data bus	The data and the instructions being transferred (via MBR / I/O BR). Its width determines how many bits move per bus cycle; with a 16-bit data bus a 32-bit instruction needs two cycles.
Control bus	Control and timing signals: memory read/write, I/O read/write, interrupt request and acknowledge, bus request/grant, clock and reset.

Q11. What is a process? Differentiate between a program and a process.

A **process** is a program in execution: an instance of a program running on a computer, the entity that can be assigned to and executed on a processor. It consists of three components: an executable **program** (code), the associated **data** and the **execution context** (all information the OS needs to manage it: PC, registers, stack, priority, etc.). When a program is ready for execution it becomes a process and can compete for CPU time, memory and I/O devices.

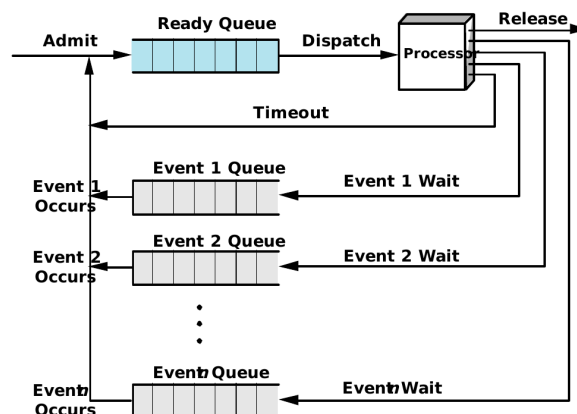
Program	Process
Passive / static entity (a set of instructions stored on disk)	Active / dynamic entity (a program in execution)
Cannot compete for resources	Competes for resources (CPU, memory, I/O)
Has only a code section	Has code section, data section, stack and program counter (plus register values)
Exists for as long as the file exists	Has a limited life: created, runs, changes states and terminates
No PCB	Has a PCB created and managed by the OS

Program	Process
One program can exist as many processes (e.g. several editor windows)	Each process is one running instance

Q12. Explain the five-state process model and describe the New, Ready, Running, Blocked/Waiting and Exit states.

The five-state model (Fig. 8) is the minimal practical model: **New, Ready, Running, Blocked/Waiting, Exit**. Transitions: Admit, Dispatch, Timeout, Event wait, Event occurs and Release.

State	Description
New	A process that has just been defined/created but not yet admitted by the OS to the pool of executable processes. It is still a program in secondary storage (the job queue); its PCB exists or is being built.
Ready	The process is in main memory and prepared to execute as soon as it is given the CPU. It waits in the <i>ready queue</i> and can compete for the CPU but is not executing.
Running	The process is currently being executed by the processor; its instructions are being run. On a uniprocessor only one process is Running at a time. It does not hold the CPU until it ends – a timeout or interrupt can send it back to Ready.
Blocked / Waiting	The process cannot continue until some event occurs (I/O completion, resource availability, message). It waits in a blocked queue in main memory; when the event occurs it moves to Ready (not directly to Running).
Exit	The process has finished or been aborted/halted. The OS releases its resources, though tables may be kept briefly for accounting.



(b) Multiple blocked queues

Fig. 12 – Queueing model with a ready queue and multiple blocked queues, one per event (Source: Chapter 3, Fig. 3.8b)

Suspended states. If every process is blocked, the OS may swap some to disk. A blocked process swapped out becomes *Blocked/Suspend*; when its event occurs it becomes *Ready/Suspend* and is later activated back to Ready.

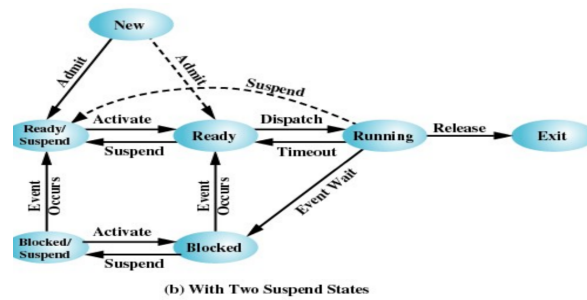


Figure 3.9 Process State Transition Diagram with Suspend States

Fig. 13 – State transition diagram with two suspend states (Source: Chapter 3, Fig. 3.9b)

Q13. What is cache memory? Why is it used in a computer system?

Cache memory is a small, fast memory placed between the processor and main memory. It contains a copy of a portion of main memory. When the processor wants a byte or word it checks the cache first; if it is not found (a *miss*), a block of memory containing the needed data is moved into the cache and delivered to the processor.

Why it is used

- In every instruction cycle the processor accesses memory several times (fetch instruction, fetch operands, store result). The processor speed is therefore limited by the memory cycle time – a **bottleneck**.
- Building all main memory with register-speed technology would be far too **expensive**.
- Because of the **principle of locality of reference** (temporal and spatial locality) programs keep re-using nearby locations, so a small fast cache gives a high hit ratio and makes memory appear almost as fast as the cache while remaining cheap and large.

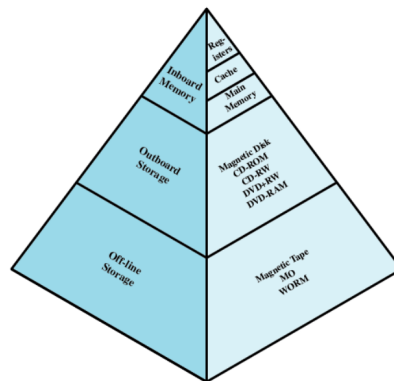


Figure 1.14 The Memory Hierarchy

Fig. 14 – The memory hierarchy (Source: Chapter 1, Fig. 1.14). Going down: lower cost per bit, larger capacity, longer access time, less frequent access.

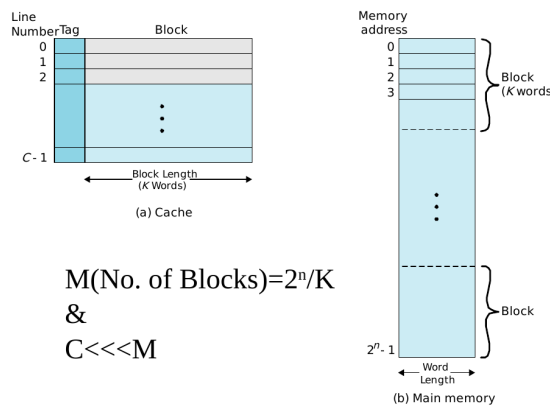


Figure 1.17 Cache/Main-Memory Structure

Fig. 15 – Cache / main-memory structure (Source: Chapter 1, Fig. 1.17)

Effect on access time (example from Chapter 1)

With a cache hit ratio of 95%, level-1 access time 0.1 μs and level-2 access time 1 μs : average access time = $0.95 \times 0.1 + 0.05 \times (0.1 + 1) = 0.095 + 0.055 = \mathbf{0.15 \mu\text{s}}$ (the slide prints “s” but the unit is μs) – close to the cache speed.

Cost versus speed (Practice Exercise 2)

Memory	Access time	Cost / bit	Cost for 1 MB (8,388,608 bits)
Main memory	1200 ns	0.001 cent	8,388.608 cents $\approx$ \$83.89
Cache technology	100 ns	0.01 cent	83,886.08 cents $\approx$ \$838.86

Cache costs 10 times more per bit but is 12 times faster, which is why only a small amount is used.

Cache design issues: cache size, block size, mapping function, replacement algorithm (e.g. LRU) and write policy (write on every update, or only when a block is replaced).

Q14. Differentiate between User-Level Threads (ULT) and Kernel-Level Threads (KLT).

Basis	User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Management	All thread management done by the application using a thread library	Done by the kernel; no thread library needed in the application
Kernel awareness	Kernel is not aware that threads exist; sees only the process	Kernel knows each thread and keeps context for the process and its threads
Scheduling	By the application/library (can be application-specific); kernel schedules the process	By the kernel on a thread basis
Thread switch	No mode switch to the kernel; fast	Needs a mode switch to the kernel; slower
Blocking system call	Blocks all threads of the process	Only the calling thread blocks; kernel can run another thread of the same process
Multiprocessor use	Cannot run threads of one process on several processors at once	Can schedule threads of one process on multiple processors simultaneously
Portability	Can run on any OS	OS-specific (e.g. Windows)
Overhead	Low	Higher (kernel involvement)

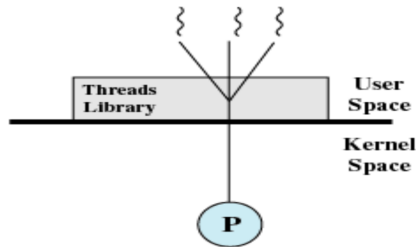


Fig. 16 – Pure user-level threads (Source: Chapter 4, Fig. 4.5a)

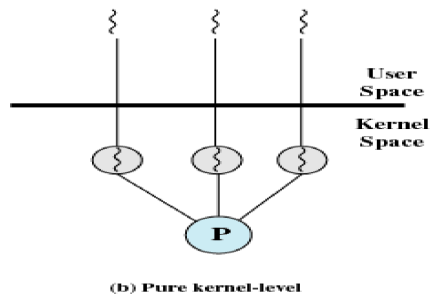


Fig. 17 – Pure kernel-level threads (Source: Chapter 4, Fig. 4.5b)

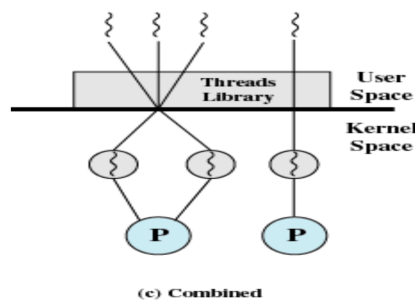


Fig. 18 – Combined approach, e.g. Solaris: thread creation in user space; bulk of scheduling and synchronisation in the application (Source: Chapter 4, Fig. 4.5c)

Q15. What is mutual exclusion? Explain the requirements that a solution to the Mutual Exclusion problem should satisfy.

Mutual exclusion means that when one process is executing in its critical section (accessing a shared resource), no other process may be in its critical section for that resource. Only one process at a time is allowed – for example, only one process at a time may send commands to the printer.

Requirements

- **Mutual exclusion must be enforced** – only one process at a time in the critical section for a resource.
- A process that **halts in its non-critical section** must do so without interfering with other processes.
- **No deadlock or starvation** – a process requiring the critical section must not be delayed indefinitely.
- A process must **not be delayed access** to a critical section when no other process is using it.
- **No assumptions** about relative process speeds or the number of processes.
- A process remains inside its critical section for a **finite time** only.

Ways to achieve it

Approach	Notes
Hardware: interrupt disabling	Simple on a uniprocessor; limits interleaving and does not work on multiprocessors
Hardware: special instructions (compare-and-swap, exchange/XCHG)	Atomic; work for any number of processes, even on multiprocessors; but busy-waiting wastes CPU time, starvation and deadlock are possible
OS / language support: semaphores, monitors, message passing	Most common; processes wait in a queue instead of spinning
Software by processes themselves	Processes coordinate with no OS or language support

Q16. What is a critical section? Why is mutual exclusion required in concurrent processes?

A **critical section** is a section of code in which a process accesses shared resources (a global variable, a file, a device) and which must not be executed by more than one process at a time.

Why mutual exclusion is needed - race condition

When concurrent processes access shared data without control, the result depends on the relative speed and interleaving of the processes – a **race condition**. Two examples from the slides:

- **The echo procedure** (Chapter 5): P1 reads a character into the shared variable *chin*; before it prints, P2 runs and overwrites *chin*. P1 then prints P2's character and P1's character is lost. The discipline “only one process in echo at a time” cures this.
- **Shared counter** (Unit 3): the shared variable is 5. P1 does $x = \text{shared}$; $x++$ and P2 does $y = \text{shared}$; $y--$. Because both read 5 before either writes back, the final value is 4 or 6 instead of 5.

Mutual exclusion is therefore required to guarantee correct results, protect data, make the output independent of execution speed and avoid errors that are very difficult to reproduce. Enforcing it creates two further problems that must be solved: **deadlock** and **starvation**.

Q17. Define deadlock. Explain the necessary conditions for deadlock.

A **deadlock** is a situation in which a set of processes is permanently blocked because each process holds a resource and is waiting for another resource held by some other process in the set; none can proceed. Example: P1 holds R1 and needs R2, while P2 holds R2 and needs R1 – both wait forever. (The slide compares it to two trains facing each other on a single track.)

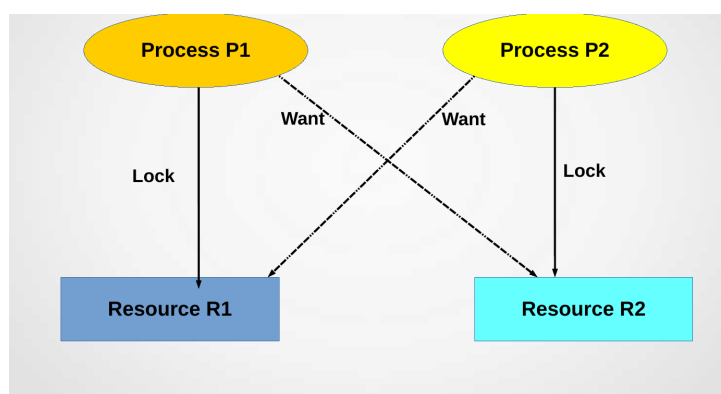


Fig. 19 – Deadlock: P1 locks R1 and wants R2; P2 locks R2 and wants R1 (Source: Unit 3, p. 46)

Four necessary conditions (all must hold together)

Condition	Explanation
1. Mutual exclusion	A resource can be used by only one process at a time (e.g. printer, memory location)
2. Hold and wait	A process holding at least one resource is waiting to acquire additional resources held by others
3. No pre-emption	A resource cannot be forcibly taken away; it is released only voluntarily by the process holding it
4. Circular wait	A closed chain of processes exists in which each process holds a resource needed by the next one ($P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_1$)

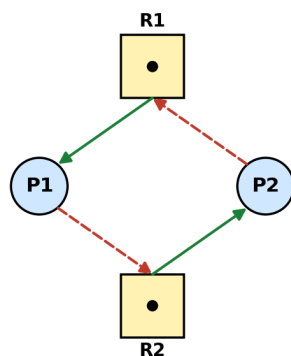
The first three conditions together make the fourth possible. If any one condition is prevented, a deadlock cannot occur.

Q18. Illustrate the Resource Allocation Graph (RAG) with a suitable diagram and explain how it can be used to identify deadlock.

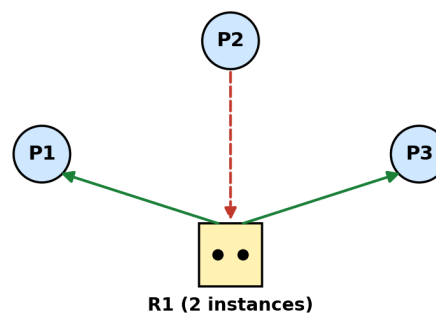
A **Resource Allocation Graph** is a directed graph that shows which resources are allocated to which processes and which processes are waiting. Vertices are **processes** (circles) and **resources** (rectangles, with a dot for each instance).

- **Request edge** $P \rightarrow R$: process P has requested resource R and is waiting (dashed).
- **Assignment edge** $R \rightarrow P$: an instance of R is allocated to P (solid).

(a) Cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$ = DEADLOCK



(b) No cycle = no deadlock



Solid arrow resource $\rightarrow$ process = ASSIGNED Dashed arrow process $\rightarrow$ resource = REQUEST Dot = one instance

Fig. 20 – Resource Allocation Graphs: (a) a cycle, which means deadlock; (b) no cycle, which means no deadlock (drawn for this document)

Using the RAG to detect deadlock

- **No cycle in the graph $\rightarrow$ no deadlock.**
- **Cycle and every resource has a single instance $\rightarrow$ deadlock** (cycle is necessary and sufficient).
In Fig. 20(a), P1 holds R1 and requests R2, P2 holds R2 and requests R1: cycle $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_1 \rightarrow P_1$, so P1 and P2 are deadlocked.
- **Cycle and resources have several instances $\rightarrow$ deadlock is possible but not certain** (cycle is necessary but not sufficient); one must check whether another process outside the cycle can release an instance.
- In Fig. 20(b) R1 has two instances, both assigned; P2 is waiting, but there is no cycle, so when P1 or P3 releases, P2 proceeds.
- Steps: draw the graph from the current allocation/request tables, search for cycles; if found, the processes on the cycle are (potentially) deadlocked and recovery (abort a process, pre-empt a resource, roll back) is needed. A wait-for graph, obtained by removing resource nodes, is used in the same way.

Q19. What is a semaphore? Explain the basic idea of semaphore operations.

A **semaphore** is a special integer variable used for signalling between processes. It may be initialised to a non-negative value and can be changed only through two **atomic** (indivisible) operations – **wait** (P, semWait) and **signal** (V, semSignal). A process that has to wait for a signal is suspended until the signal is sent. Semaphores act as a guard or lock on a shared resource to enforce mutual exclusion, avoid race conditions and synchronise processes.

```
struct semaphore {
    int count;
    queueType queue;
}

void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0)
    {
        place this process in s.queue;
        block this process
    }
}

void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0)
    {
        remove a process P from s.queue;
        place process P on ready list;
    }
}
```

Figure 5.3 A Definition of Semaphore Primitives

Fig. 21 – Definition of semaphore primitives (Source: Chapter 5, Fig. 5.3)

- **semWait(s)**: s.count is decremented; if it becomes negative, the calling process is blocked and put in the semaphore's queue; otherwise it continues.
- **semSignal(s)**: s.count is incremented; if the result is ≤ 0 (some process is waiting), one blocked process is removed from the queue and made ready.
- Using a semaphore for a critical section: initialise $s = 1$; each process does *semWait(s)*; *critical section*; *semSignal(s)*.
- Interpretation: $\text{count} \geq 0$ means the number of further processes that can pass without blocking; $\text{count} < 0$ means $|\text{count}|$ processes are blocked.

Note: Unit 3 shows a simpler busy-waiting definition, $P(S)$: while $(S \leq 0)$ do nothing; $S = S - 1$ and $V(S)$: $S = S + 1$. Busy waiting wastes CPU cycles (a “spinlock”); the queue-based version above avoids this.

Q20. Differentiate between binary and counting semaphores.

Basis	Binary semaphore	Counting (general) semaphore
Value range	Only 0 or 1	Any non-negative integer (can be greater than 1)
Purpose	Mutual exclusion for a single resource / critical section; also signalling	Controls access to a resource with several identical instances; also producer-consumer buffer counts
Initial value	1 (free) or 0	Number of available instances, e.g. $S = 4$
Meaning	1 – a process may enter the critical section; 0 – it must wait	Value = number of free instances; 0 – all busy, requester must wait
Operations	semWaitB / semSignalB (value cannot exceed 1)	semWait / semSignal (count goes up and down)
Example	Protecting a shared variable or a printer	4 printers: P1-P4 wait(S) and use them; P5 waits until one calls signal(S)

Binary: semWaitB blocks when the value is 0, otherwise sets it to 0; semSignalB wakes a blocked process or sets the value to 1. A binary semaphore is sometimes called a mutex lock (strictly, a mutex must be released by the process that locked it).

Q21. Explain the role of the Processor in a computer system and describe the functions of processor registers.

The **processor (CPU)** is the active component of the computer: it fetches instructions from memory, decodes and executes them, and thereby controls the operation of all other components. Each instruction cycle has two steps – fetch and execute – and the actions fall into four categories:

- **Processor-memory:** transfer data between processor and memory;
- **Processor-I/O:** transfer data to or from a peripheral device;
- **Data processing:** arithmetic or logic operation on data;
- **Control:** alter the sequence of execution (jump, branch).

After each instruction the processor also checks for interrupts (Q2). The processor contains a small, very fast set of **registers**:

Register group	Registers and function
User-visible registers	Can be referenced by machine language and used by all programs; they let programmers minimise main-memory references. <i>Data registers</i> hold operands/results. <i>Address registers:</i> <i>index</i> (index added to a base to form an address), <i>segment pointer</i> (segment + offset), <i>stack pointer</i> (top of stack).
Control and status registers	Used by the processor to control its operation and by privileged OS routines to control program execution. PC – address of next instruction; IR – most recently fetched instruction; PSW – condition codes/flags (positive, negative, zero, overflow), interrupt enable/disable, supervisor/user mode.
MAR and MBR	MAR specifies the address for the next read or write; MBR holds data to be written to memory or received from memory.
I/O AR and I/O BR	I/O address register specifies a particular I/O device; I/O buffer register is used for data exchange between an I/O module and the processor.

Q22. What are Program Counter (PC) and Instruction Register (IR)? Explain their roles.

	Program Counter (PC)	Instruction Register (IR)
Contains	Address of the instruction to be fetched next	The instruction most recently fetched
Role	Controls the sequence of execution. After each fetch the PC is incremented (unless a branch changes it, or an interrupt loads a new value)	Holds the instruction while the processor decodes and executes it: opcode bits say which action to perform, remaining bits give operand/address
Saved on a process switch?	Yes – stored in the PCB / pushed on the stack on an interrupt	Not needed to be saved; it is refilled by the next fetch

Example (hypothetical 16-bit machine, Chapter 1): 4 bits opcode + 12 bits address (up to 4K words).

PC=300 -> fetch 1940 (hex) into IR, PC=301. 1 = load AC from memory, address 940
PC=301 -> fetch 5941 into IR, PC=302. 5 = add contents of 941 to AC
PC=302 -> fetch 2941 into IR, PC=303. 2 = store AC into location 941

Each fetch uses the MAR (address from PC) and MBR (instruction returned), after which the instruction is moved to the IR.

Q23. Explain the difference between Memory Address Register (MAR) and Memory Buffer Register (MBR).

Basis	MAR	MBR
Holds	An address – location in memory for the next read or write	Data/instruction – value being written to memory or just read from it
Connected to	Address lines of the system bus	Data lines of the system bus
Direction	Always output from the processor to memory	Bidirectional: written by the processor (store) or by memory (load)
Typical size	Width of the address bus (decides addressable memory)	Width of the data bus / memory word
Role in fetch	PC value is copied to the MAR to say which instruction is wanted	Memory places the instruction in the MBR, which then goes to the IR

The I/O address register and I/O buffer register play the same two roles for I/O modules.

Q24. What is kernel mode? Why is it more privileged than user mode?

Kernel mode (system/control/supervisor mode) is the processor mode in which the operating-system kernel executes. In this mode privileged instructions can be executed and protected areas of memory can be accessed. See Fig. 1 and the comparison in Q1.

Why it must be more privileged

- The OS has to **control hardware and resources**: it executes privileged instructions (I/O, interrupt enable/disable, memory-management registers, changing the mode, halting).
- **Memory protection**: the area holding the OS (monitor) must not be altered by user programs; only kernel mode may change protected memory.
- **Protection among users**: a faulty or malicious user program cannot read or overwrite another process's data or take over the machine.
- **Fair resource use**: the timer (also a protected feature) prevents a job from monopolising the CPU.
- **System reliability and security**: a user-program error is confined to user mode; the kernel stays safe.

These hardware features – memory protection, timer, privileged instructions and interrupts – were introduced with simple batch systems and are the foundation of modern OS protection.

Q25. Explain multiprogramming and discuss how it improves processor utilization.

Multiprogramming means that the processor has more than one program (job) in memory to execute. In *uniprogramming* the processor must wait for each I/O instruction to complete before proceeding, so it is idle for long periods because I/O devices are slow. In multiprogramming, when one job needs to wait for I/O, the processor switches to another job. Which program runs next depends on relative priority and whether programs are waiting for I/O. After an interrupt handler completes, control may not return to the program that was running at the time of the interrupt.

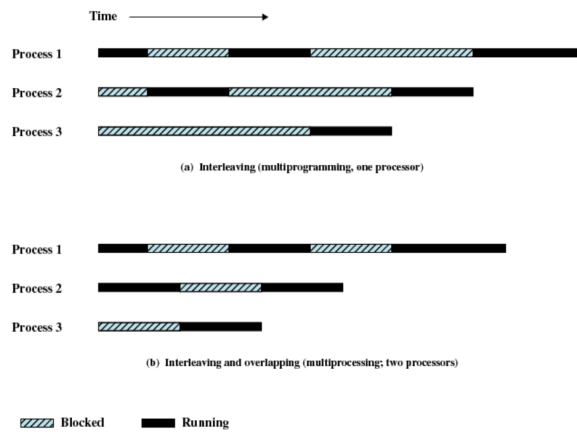


Figure 2.12 Multiprogramming and Multiprocessing

Fig. 22 – Interleaved execution of three processes on one processor – blocked (hatched) periods of one process are used to run others (Source: Chapter 2, Fig. 2.12a)

How it improves utilisation

- CPU idle time during I/O waits is filled with useful work of other jobs, so CPU and I/O devices stay busy together.
- Throughput rises – more jobs complete per unit of time; average resource utilisation (CPU, memory, I/O) is better.
- It is the basis of time sharing, where the processor's time is shared among many interactive users.
- Illustration (standard model, not from the slides): if each process spends 80% of its time waiting for I/O, one process keeps the CPU busy only 20% of the time; with three processes utilisation $\approx 1 - 0.8^3 \approx 49\%$, and with ten $\approx 89\%$.

Requirements: enough memory to hold several programs, memory protection, scheduling, and interrupt support. The cost is more complex OS design.

Q26. What is the kernel of an operating system? Explain its role.

The **kernel** (also called the **nucleus**) is the portion of the operating system that is kept in main memory and contains the most frequently used functions. It runs in kernel (privileged) mode and is the core that sits directly on top of the hardware.

Role of the kernel

- **Process and thread management** – creation, scheduling, dispatching, switching and termination.
- **Memory management** – allocation, protection, virtual memory.
- **Interrupt and exception handling** – first-level handling of interrupts, traps and system calls.
- **Device (I/O) management** – via device drivers, provides access to hardware.
- **File management and interprocess communication/synchronisation.**
- Provides the system-call interface through which user programs request OS services, and enforces protection and security.

Examples: In Windows the kernel-mode components are the Executive, Kernel, HAL, device drivers and window/graphics system. In UNIX the OS surrounding the hardware is called the system kernel, with shell and utilities around it. Kernels may be layered/monolithic or microkernel-based (Q32).

Q27. What is process switching? Explain the role of the program counter and processor registers during a process switch.

Process switching (context switch) is the act of the OS taking the processor away from the running process and giving it to another process. The dispatcher performs this.

When a switch can occur

Mechanism	Cause
Clock interrupt	Process has used up its maximum time slice
I/O interrupt	An I/O operation finished, so a blocked process becomes ready
Memory fault	Referenced address is in virtual memory and must be brought into main memory
Trap	Error or exception occurred; the process may be moved to Exit
Supervisor call	Process requests an OS service such as file open

Steps

- 1. Save the context of the processor, including the **program counter** and other registers.
- 2. Update the PCB of the process currently in the Running state (new state, accounting).
- 3. Move that PCB to the appropriate queue – ready, blocked or ready/suspend.
- 4. Select another process for execution (scheduler).
- 5. Update the PCB of the selected process (state = Running) and the memory-management data structures.
- 6. Restore the context of the selected process – load its saved PC and register values.

Role of the PC and registers

The **PC** holds the address of the next instruction. Saving it in the PCB means the interrupted process can later continue at exactly the same instruction; loading the PC of the new process makes the processor fetch from that process's code. The **general-purpose and status registers** (including PSW, condition codes and stack pointer) hold the intermediate results and execution status. They are all saved, because the new process will overwrite them, and restored later so the process sees no change. A *mode switch* (user→kernel on an interrupt) is lighter than a full process switch, because the latter also changes the PCB state, queues and memory mapping.

Q28. What events can cause a process to change its state? Explain with suitable examples.

Event	Transition	Example
A new process is created / job admitted	New → Ready	User starts a program; long-term scheduler loads the job
Scheduler selects the process	Ready → Running (Dispatch)	CPU becomes free and the short-term scheduler picks the next process
Time slice over (timer interrupt)	Running → Ready	Round-robin quantum expires
Higher-priority process arrives in the ready queue	Running → Ready (pre-emption)	An urgent real-time task becomes ready
Process requests I/O or a resource / waits for an event	Running → Blocked	Reading a file or keyboard input; waiting for a printer
I/O completes / resource released / event occurs	Blocked → Ready	Disk read finishes; interrupt wakes the process
Process ends or is aborted; error or exception	Running → Exit	Program finishes, or divides by zero (trap)
Memory is short – all processes blocked	Blocked → Blocked/Suspend; Ready → Ready/Suspend	OS swaps a process to disk
Event for suspended process occurs; memory is free	Blocked/Suspend → Ready/Suspend → Ready (Activate)	Swapped-out process is brought back
Any hardware interrupt	Running → Ready/Blocked	Handler runs and the scheduler is invoked

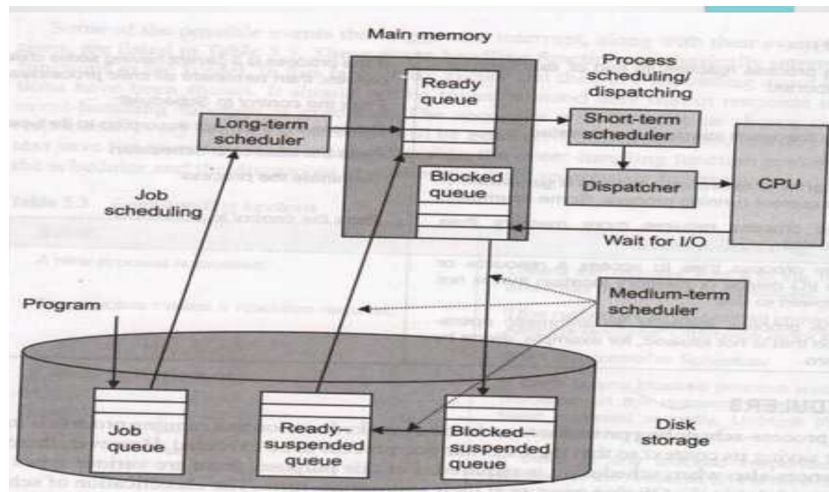


Fig. 23 – Schedulers and queues: long-term (job) scheduling, short-term (process) scheduling and medium-term scheduling (Source: Unit 2, p. 48)

Q29. What are User-Level Threads? Explain their advantages and limitations.

User-level threads (ULT) are threads whose management is done entirely by the application through a threads library in user space. The kernel is not aware of their existence; it schedules the containing process as a single unit (Fig. 16).

Advantages	Limitations
Saves the overhead of two mode switches (user→kernel and back); thread switching does not need kernel mode	When one thread makes a blocking system call, the entire process (all its threads) is blocked
Scheduling can be application-specific	A multithreaded application cannot take advantage of multiprocessing – the kernel assigns a process to only one processor at a time
Can run on any operating system – no kernel changes, only a library	No kernel support: no pre-emption of threads by the kernel; kernel scheduling decisions are blind to thread needs
Fast creation and switching	

Remedy - jacketing: converts a blocking system call into a non-blocking one. Instead of calling the system I/O routine directly, the thread calls an application-level I/O jacket routine that checks whether the device is busy and lets another thread run.

Q30. What are Kernel-Level Threads? Explain their advantages and limitations.

Kernel-level threads (KLT) are threads managed directly by the operating-system kernel. No thread library is needed; the kernel knows about each thread, maintains context information for the process and its threads, and schedules on a thread basis. Windows uses this approach (Fig. 17).

Advantages	Limitations
The kernel can simultaneously schedule multiple threads of the same process on multiple processors	Transfer of control from one thread to another within the same process requires a mode switch to the kernel – slower than ULT switching
If one thread blocks, the kernel can schedule another thread of the same process	Creating and managing threads costs more (system calls, kernel data structures)
Kernel routines themselves can be multithreaded	Less portable; depends on OS support

Combined approach (Solaris, Fig. 18): thread creation in user space, with the bulk of scheduling and synchronisation within the application, mapped onto kernel threads – giving the advantages of both.

Q31. What is Symmetric Multiprocessing (SMP)? List its important characteristics.

An **SMP** is a stand-alone computer system with two or more similar processors that share the same main memory and I/O facilities and are controlled by one integrated operating system. The kernel can execute on any processor, and typically each processor does its own scheduling from the pool of available processes or threads.

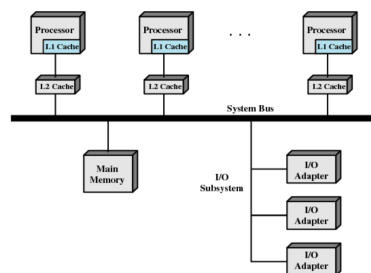


Figure 4.9 Symmetric Multiprocessor Organization

Fig. 24 – Symmetric multiprocessor organisation: processors with private L1/L2 caches, shared main memory and I/O over a system bus (Source: Chapter 4, Fig. 4.9)

Characteristics

- Two or more similar processors of comparable capability.
- They share the same main memory and I/O facilities, interconnected by a bus or other scheme, so memory access time is about the same for each processor.
- All processors share access to I/O devices, through the same or different channels leading to the same devices.
- All processors can perform the same functions (hence “symmetric”).
- The system is controlled by an integrated operating system that provides interaction between processors and programs at the job, task, file and data-element levels.

Advantages over a uniprocessor: performance (work done in parallel), availability (one failure does not halt the machine), incremental growth (add a processor) and scaling (range of products). **Design issues:** simultaneous concurrent processes/threads, scheduling, synchronisation, memory management, reliability and fault tolerance, and the **cache-coherence problem** (a word changed in one cache must be made visible to other caches – solved in hardware).

Q32. What is a microkernel? Explain the basic idea and benefits of microkernel organization.

A **microkernel** is a small operating-system core that contains only the essential core functions. Services traditionally inside the OS – device drivers, file systems, virtual-memory manager, windowing system and security services – are moved out into external subsystems running as user-mode server processes. The microkernel itself typically provides low-level memory management (mapping virtual pages to physical frames), interprocess communication (message passing), I/O and interrupt management, and basic scheduling.

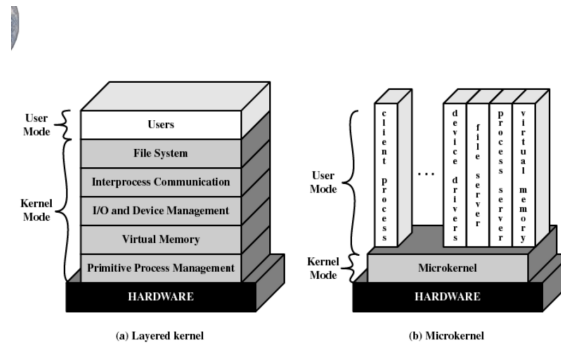


Figure 4.10 Kernel Architecture

Fig. 25 – (a) Layered kernel versus (b) microkernel architecture (Source: Chapter 4, Fig. 4.10)

Benefits

Benefit	Explanation
Uniform interface	Every request is a message; no distinction between kernel-level and user-level services
Extensibility	New services can be added easily
Flexibility	Features can be added and existing ones removed
Portability	Only the microkernel must be changed to move to a new processor
Reliability	Modular design; the small kernel can be rigorously tested; a failing server does not crash the kernel
Distributed system support	Messages can be sent without knowing which machine is the target
Object-oriented OS	Components are objects with clearly defined interfaces

Windows uses a *modified* microkernel architecture (not pure): many system functions outside the microkernel still run in kernel mode.

Q33. What is concurrency? Explain its relationship with mutual exclusion and synchronization.

Concurrency means that several processes or threads are in progress at the same time – interleaved on one processor (multiprogramming) or overlapped on several (multiprocessing, distributed processing). It arises in multiple applications, structured applications (a set of concurrent processes) and in the OS itself (a set of processes/threads). The design issues are communication among processes, sharing of and competition for resources, synchronisation of activities and allocation of processor time.

Relationship

- Concurrent processes **share resources** (global variables, files, devices). Because relative execution speeds are unpredictable, uncontrolled access causes race conditions and errors that are hard to reproduce.
- Mutual exclusion** is the control needed when processes *compete* for a resource: only one at a time may be in the critical section.
- Synchronisation** is the control needed when processes *cooperate*: one process must wait for a signal or data (e.g. a consumer waits until a producer has placed an item; message passing needs send/receive coordination).
- Both are provided by the same tools – semaphores, monitors and message passing – and mis-using them leads to **deadlock** and **starvation**.

Process interaction	Influence / control problem
Processes unaware of each other (competition)	Mutual exclusion, deadlock, starvation
Processes indirectly aware (sharing)	Mutual exclusion, deadlock, starvation, data coherence
Processes directly aware (cooperation by communication)	Deadlock, starvation, synchronisation

Q34. What is starvation? Explain how it differs from deadlock in concurrent systems.

Starvation occurs when a process waits indefinitely for some resource while other processes are actually using it and making progress. A low-priority process may be postponed forever because high-priority processes keep getting the resource.

Basis	Deadlock	Starvation
Meaning	A set of processes is blocked, each waiting for a resource held by another in the set; none can proceed	One (or few) process waits too long because others always get the resource; the system as a whole progresses
Progress	No process in the set makes progress	Other processes continue to make progress
Waiting	Infinite waiting (circular)	Long waiting, possibly (but not necessarily) infinite
Cause	Mutual exclusion + hold and wait + no pre-emption + circular wait together	Uncontrolled priority/resource-allocation policy (e.g. SSTF, priority scheduling)
Relation	Every deadlock involves starvation of the processes in it	Not every starvation is a deadlock
Solution	Prevention, avoidance, detection and recovery	Aging (increase priority with waiting time), fair/FIFO policies

Q35. Explain the semWait and semSignal operations of a semaphore.

```

struct semaphore { int count; queueType queue; }

void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        place this process in
        s.queue; block this process
    }
}

void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        remove a process P from
        s.queue; place P on ready list
    }
}

```

- **semWait** (also called P, wait, down): decrements the semaphore value. If the value becomes negative the process executing semWait is **blocked**; otherwise it continues. It is the *entry* criterion of a critical section.
- **semSignal** (V, signal, up): increments the value. If the result is ≤ 0 , a process blocked by an earlier semWait is **unblocked** (moved to the ready list). It is the *exit* criterion.
- Both operations are atomic: no interleaving with any other operation on the same semaphore.
- A negative count equals the number of processes waiting.

Trace - mutual exclusion with $s = 1$ (three processes)

Step	Operation	s.count	Result
1	A: semWait(s)	0	A enters critical section
2	B: semWait(s)	-1	B blocked (queue: B)
3	C: semWait(s)	-2	C blocked (queue: B, C)
4	A: semSignal(s)	-1	B released → ready, enters critical section
5	B: semSignal(s)	0	C released → ready
6	C: semSignal(s)	1	No one waiting; semaphore free

Caution: reversing the order of semWait operations (for example in the producer-consumer solution) can cause deadlock, which is why correct design with semaphores is difficult.

Q36. Differentiate between deadlock prevention, deadlock avoidance, and deadlock detection.

Basis	Prevention	Avoidance	Detection (and recovery)
Idea	Ensure that at least one of the four necessary conditions can never hold	Grant a request only if the resulting state is safe, using advance information	Allow deadlock to occur, find it, then recover
Information needed	None about future requests	Maximum needs of each process, available and allocated resources	Current allocation and request status
When applied	Design time (rules for requesting resources)	At every resource request	Periodically or when processes stop progressing
Methods	Mutual exclusion: make resources sharable. Hold & wait: request all at once / release before requesting. No pre-emption: pre-empt held resources. Circular wait: number resources and request in order	Banker's algorithm, safe-state check, RAG with claim edges	Wait-for graph / RAG cycle search, detection algorithm; recovery by aborting processes, pre-empting resources, rollback
Drawbacks	Very restrictive; low resource utilisation; not all resources can be pre-empted/shared	Needs prior knowledge of maximum demands; run-time overhead; processes may be delayed	Deadlock does occur; recovery is costly, may lose work
Strictness	Most conservative	Intermediate	Most liberal

Q37. What is a safe state and what is an unsafe state in deadlock avoidance?

Safe state: a state in which the system can allocate resources to every process (up to its maximum need) in *some order* and still avoid deadlock. Such an order is a **safe sequence** $\langle P_1, P_2, \dots, P_n \rangle$: for each P_i the resources it still needs can be satisfied by the currently available resources plus those held by all earlier P_j ($j < i$), which will finish and release them.

Unsafe state: a state for which *no* safe sequence exists. An unsafe state is **not necessarily a deadlock**, but it may lead to one, depending on how processes request resources. A deadlocked state is always unsafe.

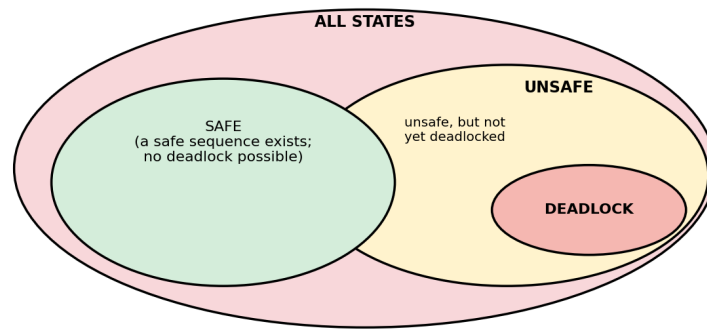


Fig. 26 – Relation between safe, unsafe and deadlock states (drawn for this document)

Avoidance rule: when a process requests resources, the OS pretends to grant them and runs the safety test; if the resulting state is safe the request is granted, otherwise the process must wait. Example (from Chapter 6): the initial state of the 6-process system is **safe** with sequence $P1 \rightarrow P3 \rightarrow P4 \rightarrow P2 \rightarrow P5 \rightarrow P0$, but granting $P5$ the request (3,2,3,3) would give an **unsafe** state, so it is refused (see Q38).

Q38. Explain the Banker's Algorithm for deadlock avoidance and describe the information required to determine whether a state is safe.

The **Banker's algorithm** (Dijkstra) is a deadlock-avoidance algorithm. Like a banker who never lends money so that he cannot satisfy all customers' needs, the OS grants a resource request only if the system stays in a safe state afterwards. Each process must declare its maximum resource need in advance.

Information required (n processes, m resource types)

Data structure	Meaning
Available [m]	Number of free instances of each resource type
Max [n × m]	Maximum demand of each process for each resource
Allocation [n × m]	Resources currently held by each process
Need [n × m]	Remaining need: Need = Max - Allocation

Safety algorithm

1. Let $Work = Available$ and $Finish[i] = false$ for all i .
2. Find a process i with $Finish[i] = false$ and $Need[i] \leq Work$. If none exists, go to step 4.
3. $Work = Work + Allocation[i]$; $Finish[i] = true$; go to step 2.
4. If $Finish[i] = true$ for all i , the state is **safe**; otherwise it is **unsafe**.

Resource-request algorithm (process P_i requests $Request[i]$)

1. If $Request \leq Need[i]$, go on; otherwise error (process exceeded its maximum claim).
2. If $Request \leq Available$, go on; otherwise P_i must wait.
3. Pretend to allocate: $Available = Available - Request$; $Allocation[i] += Request$; $Need[i] -= Request$.
4. Run the safety algorithm. If safe → grant; if unsafe → restore the old state and make P_i wait.

Worked example (Chapter 6): 6 processes, resources A, B, C, D

Total resources $A = 15, B = 6, C = 9, D = 10$; Available = (6, 3, 5, 4).

Process	Allocation (A B C D)	Max (A B C D)	Need = Max - Alloc
P0	2 0 2 1	9 5 5 5	7 5 3 4
P1	0 1 1 1	2 2 3 3	2 1 2 2

Process	Allocation (A B C D)	Max (A B C D)	Need = Max - Alloc
P2	4 1 0 2	7 5 4 4	3 4 4 2
P3	1 0 0 1	3 3 3 2	2 3 3 1
P4	1 1 0 0	5 2 2 1	4 1 2 1
P5	1 0 1 1	4 4 4 4	3 4 3 3

(b) Safety test starting with Work = (6,3,5,4):

Step	Process	Need	Work before	Released	Work after
1	P1	(2,1,2,2)	(6,3,5,4)	(0,1,1,1)	(6,4,6,5)
2	P3	(2,3,3,1)	(6,4,6,5)	(1,0,0,1)	(7,4,6,6)
3	P4	(4,1,2,1)	(7,4,6,6)	(1,1,0,0)	(8,5,6,6)
4	P2	(3,4,4,2)	(8,5,6,6)	(4,1,0,2)	(12,6,6,8)
5	P5	(3,4,3,3)	(12,6,6,8)	(1,0,1,1)	(13,6,7,9)
6	P0	(7,5,3,4)	(13,6,7,9)	(2,0,2,1)	(15,6,9,10)

P0 could not run first ($7 > 6$ for A) but can after the others finish. All six finish, so the state is **SAFE** with safe sequence **P1 → P3 → P4 → P2 → P5 → P0**; Work returns to the total (15,6,9,10).

(c) Request (3,2,3,3) from P5: Request $\leq$ Need(P5) = (3,4,3,3) ✓; Request $\leq$ Available (6,3,5,4) ✓. Pretend to allocate: Available = (3,1,2,1), Allocation(P5) = (4,2,4,4), Need(P5) = (0,2,0,0). Safety test with Work = (3,1,2,1): P5 needs B = 2 > 1; P1 needs D = 2 > 1; P3 needs B = 3 > 1; P4 needs A = 4 > 3; P2 and P0 also exceed Work. No process can finish, so the state is **UNSAFE** and the request must **NOT be granted** – P5 waits.

Key idea: having enough Available resources is not sufficient; a request is granted only if the resulting state remains safe.

Notes on the source material

- The five unit decks (Unit 4 on memory management and Unit 5 on file/disk management) were read but contain nothing needed for these 38 questions.
- Chapter 1 slide on average access time prints the unit as “s”; the data are in μs , so the result is 0.15 μs .
- Unit 3 describes a “mutex semaphore” as one that any process may unlock; in standard usage a mutex must be released by the process that locked it, whereas a binary semaphore can be signalled by another process.
- The three-bus split (address/data/control) in Q10, the CPU-utilisation formula in Q25 and the general meaning of Banker's-algorithm terms in Q38 are standard textbook material and are not spelled out on the slides.